

Inigo Quilez :: articles :: normals for an SDF - 2015



Intro

And SDF is a Signed Distance Function, and in the context of computer graphics, it's usually employed to rapidly **raymarch geometry and scenes**. When doing lighting on such scenes, or collision detection, having access to the surface normals of the geometry is necessary. The surfaces that emerge from an SDF $f(\mathbf{p})$, where $\mathbf{p}$ is a point in space, are given by a particular iso-surface, normally the $f(\mathbf{p}) = 0$ isosurface. Computing the normal $\mathbf{n}$ of that isosurfaces can be done through the **gradient** of the SDF at points located on the surface.

$$\mathbf{n} = \text{normalize}(\nabla f(\mathbf{p}))$$

Remember that the gradient of a scalar field is always perpendicular to the iso-lines or iso-surfaces described by the scalar field, and since the normals to a surface need to be perpendicular, the normals must align with the gradient.



Surface normals filtered for antialiasing, computed with the tetrahedron technique below
(realtime version: <https://www.shadertoy.com/view/ldScDh>)

There are multiple ways to compute such gradient. Some are numerical, others are analytical, all with different advantages and disadvantages. This article is about numerically computing them, which requires the least code writing, but might not be the fastest or most accurate. Still, its simplicity makes it the most popular way to compute normals in realtime raymarched demos and games.

Classic technique - forward and central differences

In its simplest form, we take the definition of gradient.

$$\nabla f(p) = \left\{ \frac{df(p)}{dx}, \frac{df(p)}{dy}, \frac{df(p)}{dz} \right\}$$

Those partial derivatives can be computed with small differences, as we know from the definition of derivative. For example,

$$\frac{df(p)}{dx} \approx \frac{f(p + \{h, 0, 0\}) - f(p)}{h}$$

where h is as small as possible. This is called forward differentiation, since it takes the point under consideration $p=(x,y,z)$ and evaluates f at $p'=(x+h,y,z)$, a point in the positive x direction. Backwards and central differences are also possible, which take the following forms:

$$\frac{df(p)}{dx} \approx \frac{f(p) - f(p - \{h, 0, 0\})}{h}$$

$$\frac{df(p)}{dx} \approx \frac{f(p + \{h, 0, 0\}) - f(p - \{h, 0, 0\})}{2h}$$

Actually central differences are often preferred since they don't favor neither positive nor negative axes, which is important in order to make sure the normals follow the surface without any directional bias. Note the division by two times h in the central differences method, which is the distance between the function sampling

points.

Using central differences, the normal therefore takes the following form

$$n = \text{normalize}(\{f(p + \{h, 0, 0\}) - f(p - \{h, 0, 0\}), \\ f(p + \{0, h, 0\}) - f(p - \{0, h, 0\}), \\ f(p + \{0, 0, h\}) - f(p - \{0, 0, h\})\})$$

Note how the division by $2h$ can be simplified away since the normalization will rescale the normal anyways to unit length. When encoding this in a program, it looks like this:

```
vec3 calcNormal( in vec3 p ) // for function f(p)
{
    const float eps = 0.0001; // or some other value
    const vec2 h = vec2(eps,0);
    return normalize( vec3(f(p+h.xyy) - f(p-h.xyy),
                          f(p+h.yxy) - f(p-h.yxy),
                          f(p+h.yyx) - f(p-h.yyx) ) );
}
```

This form can be found in many raymarching demos in places like Shadertoy or the demoscene productions. However, if evaluating $f(p)$ six times becomes too expensive, one can use forward differences instead

$$n = \text{normalize}(\{f(p + \{h, 0, 0\}) - f(p), \\ f(p + \{0, h, 0\}) - f(p), \\ f(p + \{0, 0, h\}) - f(p)\})$$

since it only requires four evaluations instead of six. Even though the following is not true due to numerical reasons, in some special cases we can assume $f(p)$ is zero though, since by definition we are looking for the zero isosurface, and get away with the following three evaluations:

$$n = \text{normalize}(\{f(p + \{h, 0, 0\}), \\ f(p + \{0, h, 0\}), \\ f(p + \{0, 0, h\})\})$$

Tetrahedron technique

There's a nice alternative to the direct gradient definition technique that is based also on central differences, which means the lighting done on the normals will follow the surfaces without any shift, that uses only four evaluations instead of six, making it as efficient as the forward differences. I first saw this technique used by Paulo Falcao in Pouet around 2008 and then later by Paul Malin in Shadertoy. I have embraced it in most of my shaders since then, and it looks like this:

```
vec3 calcNormal( in vec3 ) // for function f(p)
{
    const float h = 0.0001; // replace by an appropriate value
    const vec2 k = vec2(1,-1);
    return normalize( k.xyy*f( p + k.xyy*h ) +
                     k.yyx*f( p + k.yyx*h ) +
                     k.yxy*f( p + k.yxy*h ) +
                     k.xxx*f( p + k.xxx*h ) );
}
```

The reason this works is the following:

The four sampling points are arranged in a tetrahedron with vertices $\mathbf{k0} = \{1, -1, -1\}$, $\mathbf{k1} = \{-1, -1, 1\}$, $\mathbf{k2} = \{-1, 1, -1\}$ and $\mathbf{k3} = \{1, 1, 1\}$. Evaluating the sum

$$m = \sum_i k_i f(p + h k_i)$$

on those four vertices produces some nice cancellations that results in

$$m = \sum_i k_i (f(p + h k_i) - f(p))$$

which are four **directional derivatives**, meaning we can rewrite the sum as

$$m = \sum_i k_i \nabla_{k_i} f(p) = \sum_i k_i (k_i \cdot \nabla f(p))$$

We can now proceed to rewriting this using matrices, or we can keep looking at one component at a time. Let's do the later. For $\mathbf{x}$, we get

$$m_x = \sum_i k_{ix} \nabla_{k_i} f(p) = \nabla f(p) \cdot \sum_i k_{ix} k_i = \nabla f(p) \cdot \{4, 0, 0\}$$

(this makes use of the fact that the dot product is a linear operator). The results are similar for the **y** and **z** components, meaning that

$$m = 4 \nabla f(p)$$

which after normalization gives us our normal at the zero-isosurface.

An important implementation detail

I hope I can obsolete this paragraph some day in the future, but as of today, your development platform might decide to take the above code and inline the four calls to `f()` inside `calcNormal()`. If `f()` is long, as it often is, that can create problems with the number of instructions available in your platform, and often times it can crash the shader compiler (often happens in WebGL). One trick to prevent the compiler from inlining `f()` is to make sure the loop depends on some input uniform that the compiler doesn't know about. Thomas Hooper thought of this trick first and has since become pretty common among the Shadertoy community, since it helps both prevent the crashes and also get the compilation times down by a lot. This a variation of his trick made by Clément Baticle (a.k.a. Klems):

```
vec3 calcNormal( in vec3 p ) // for function f(p)
{
    const float h = 0.0001; // replace by an appropriate value
    #define ZERO (min(iFrame,0)) // non-constant zero
    vec3 n = vec3(0.0);
    for( int i=ZERO; i<4; i++ )
    {
        vec3 e = 0.5773*(2.0*vec3((((i+3)>>1)&1),((i>>1)&1),(i&1))-1.0);
        n += e*map(pos+e*h).x;
    }
    return normalize(n);
}
```

Cetral differences for Terrains

When rendering raymarched terrains we usuaully have a heightmap of the form $\mathbf{g}(\mathbf{x}, \mathbf{z})$ that defines it through $y = \mathbf{g}(\mathbf{x}, \mathbf{z})$. This can be rewritten as $\mathbf{f}(\mathbf{x}, \mathbf{y}, \mathbf{z}) = y - \mathbf{g}(\mathbf{x}, \mathbf{z}) = 0$, which means that while not a distance field it is still a scalar field, so we can still use all the gradient logic in order to comptue its normal. By doing so, we get

$$\begin{aligned}\frac{\partial f}{\partial x} &= -\frac{\partial g}{\partial x} \\ \frac{\partial f}{\partial y} &= 1 \\ \frac{\partial f}{\partial z} &= -\frac{\partial g}{\partial z}\end{aligned}$$

which means that the normal can be computed simply as

```
vec3 getNormal( in vec2 p ) // for terrain g(p)
{
    const float eps = 0.0001; // or some other value
    const vec2 h = vec2(eps,0);
    return normalize( vec3( g(p-h.xy) - g(p+h.xy),
                          2.0*h.x,
                          g(p-h.yx) - g(p+h.yx) ) );
}
```

Beware that because we are using central differences and the distance between the sampling points is $2\mathbf{h}$, removing the division by $2\mathbf{h}$ means we need to multiply the y component (which is 1) by $2\mathbf{h}$. This is the code that you can also find in [article about terrain marching](#). You can arrive to the same formula if you consider the four little triangles that you can create if you were polygonalizing the terrain field with triangles of size $\mathbf{h}$, and were computing normals for those triangles and then averaging them, as you would be doing for a mesh.

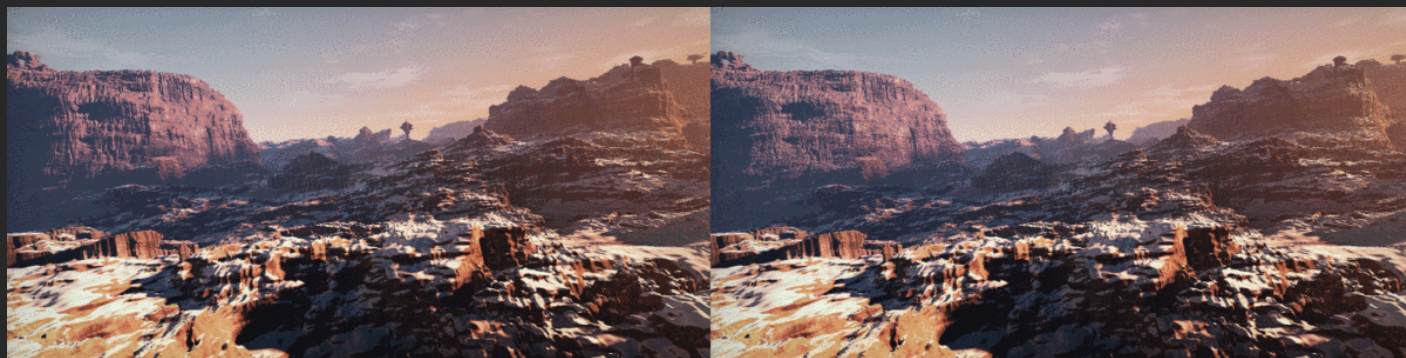
Some words about sampling and aliasing

You might have noticed that so far we haven't paid any attention to the value of $\mathbf{h}$. In theory, it needs to be as small as possible in order for the components of the gradient to properly approximate the spatial derivatives. Of course too small values will introduce numerical errors, so in practice there's a limit to how small $\mathbf{h}$ can be.

However, there's another important consideration when it comes to picking a value - geometry detail and aliasing. Indeed, when taking our central differences at a point in space, we should consider how far it is from camera, or more exactly, what's the footprint of the current pixel being rendered into world space at the sampling point. The idea is that we want to know how tiny do geometrical details get when projected to the screen in the neighborhood of the sampling point - or equivalently, how big is the pixel footprint compared to the geometrical detail in the area. We want this information so that we can do Level of Detail (LoD) in the raymarched geometry - we don't need to compute SDFs for objects/features that are really far in the distance and get too tiny due to perspective projection. This is not only a performance optimization, but more importantly, an image quality issue. By ensuring that we do not consider and sample geometry that is not big enough to be reliably captured by a ray representing the whole pixel, we are effectively implementing a type filtering (band limiting) that prevents aliasing.

Now, under such circumstances, one needs to make sure that the h used to sample the gradients is also about the size of the pixel footprint (or the size of the biggest detail we decided to reject as part of the LoD system). This is going to be something proportional to the distance from the sampling point to the camera, often referred as t in raytracers and raymarchers.

The effect is very dramatic, as can be seen in the animated GIF below. Look to the distant cliff walls on the left side of the image:



On the left you can see the stabilized/filtered normals computed by choosing h proportionally to the pixel footprint. To the right is the naive version that uses a constant h (0.001 in this case, making it bigger makes the image lose detail in the foreground). The difference is pretty big.